

Advanced programming Assignment

ROLL NO: CSB24065

NAME: Gourisankar Bhuyan

Session: Spring

Year:2026

Question:

Design a system in java/python for processing customer orders in an e-commerce platform.

An order system should support:

Multiple payment methods (Credit Card, UPI, Wallet, etc.)

Multiple notification channels (Email, SMS, Push)

Different order types (Regular Order, Discounted Order, Priority Order)

Ability to store order data using different storage mechanisms (Database, File, etc.) Design Constraints (Must Apply SOLID Principles)

Your design must satisfy the SOLID principles as follows:

1. Single Responsibility Principle (SRP): Each class should have a single responsibility (e.g., order logic, payment processing, notification, storage should be separate)
2. Open/Closed Principle (OCP): You should be able to add: New payment methods New notification types Without modifying existing classes
3. Liskov Substitution Principle (LSP): All subclasses (e.g., payment types, order types) should work correctly when used through their base type, No subclass should break expected behavior
4. Interface Segregation Principle (ISP): Avoid large interfaces, Design small, role-specific interfaces (e.g., don't force all classes to implement unused methods)
5. Dependency Inversion Principle (DIP): High-level classes (e.g., OrderService) must depend on abstractions, not concrete implementations, Use dependency injection.

Your system should:

Create an order, Process payment using a selected payment method, Send notification after successful order, Save order details using a storage mechanism;

Answer:

```
from abc import ABC, abstractmethod
```

```
class Order(ABC):
```

```
    def __init__(self, order_id, customer_name, amount):
```

```
        self.order_id = order_id
```

```
        self.customer_name = customer_name
```

```
        self.amount = amount
```

```
    @abstractmethod
```

```
    def get_total(self):
```

```
        pass
```

```
class RegularOrder(Order):
```

```
    def get_total(self):
```

```
        return self.amount
```

```
class DiscountedOrder(Order):
```

```
    def __init__(self, order_id, customer_name, amount, discount):
```

```
        super().__init__(order_id, customer_name, amount)
```

```
        self.discount = discount
```

```
    def get_total(self):
```

```
        return self.amount - self.discount
```

```
class PriorityOrder(Order):
    def __init__(self, order_id, customer_name, amount, priority_fee):
        super().__init__(order_id, customer_name, amount)
        self.priority_fee = priority_fee

    def get_total(self):
        return self.amount + self.priority_fee
```

```
class PaymentMethod(ABC):
    @abstractmethod
    def pay(self, amount):
        pass
```

```
class CreditCardPayment(PaymentMethod):
    def pay(self, amount):
        print(f"Payment of Rs.{amount} done using Credit Card")
```

```
class UPIPayment(PaymentMethod):
    def pay(self, amount):
        print(f"Payment of Rs.{amount} done using UPI")
```

```
class WalletPayment(PaymentMethod):
    def pay(self, amount):
        print(f"Payment of Rs.{amount} done using Wallet")
```

```
class NotificationService(ABC):
```

```
@abstractmethod
def send_notification(self, message):
    pass
```

```
class EmailNotification(NotificationService):
    def send_notification(self, message):
        print(f"EMAIL: {message}")
```

```
class SMSNotification(NotificationService):
    def send_notification(self, message):
        print(f"SMS: {message}")
```

```
class PushNotification(NotificationService):
    def send_notification(self, message):
        print(f"PUSH: {message}")
```

```
class Storage(ABC):
    @abstractmethod
    def save(self, order):
        pass
```

```
class DatabaseStorage(Storage):
    def save(self, order):
        print(f"Order {order.order_id} saved in Database")
```

```

class FileStorage(Storage):
    def save(self, order):
        print(f"Order {order.order_id} saved in File")

class OrderService:
    def __init__(self, payment_method, notification_service, storage):
        self.payment_method = payment_method
        self.notification_service = notification_service
        self.storage = storage

    def place_order(self, order):
        total = order.get_total()

        print("\n----- ORDER DETAILS -----")
        print("Order ID:", order.order_id)
        print("Customer:", order.customer_name)
        print("Total Amount:", total)

        self.payment_method.pay(total)

        self.notification_service.send_notification(
            f"Order {order.order_id} placed successfully"
        )

        self.storage.save(order)

        print("Order Processing Completed\n")

order1 = RegularOrder(101, "Gourisankar", 5000)

```

```
order2 = DiscountedOrder(102, "Rahul", 4000, 500)
order3 = PriorityOrder(103, "Ankit", 3000, 300)
```

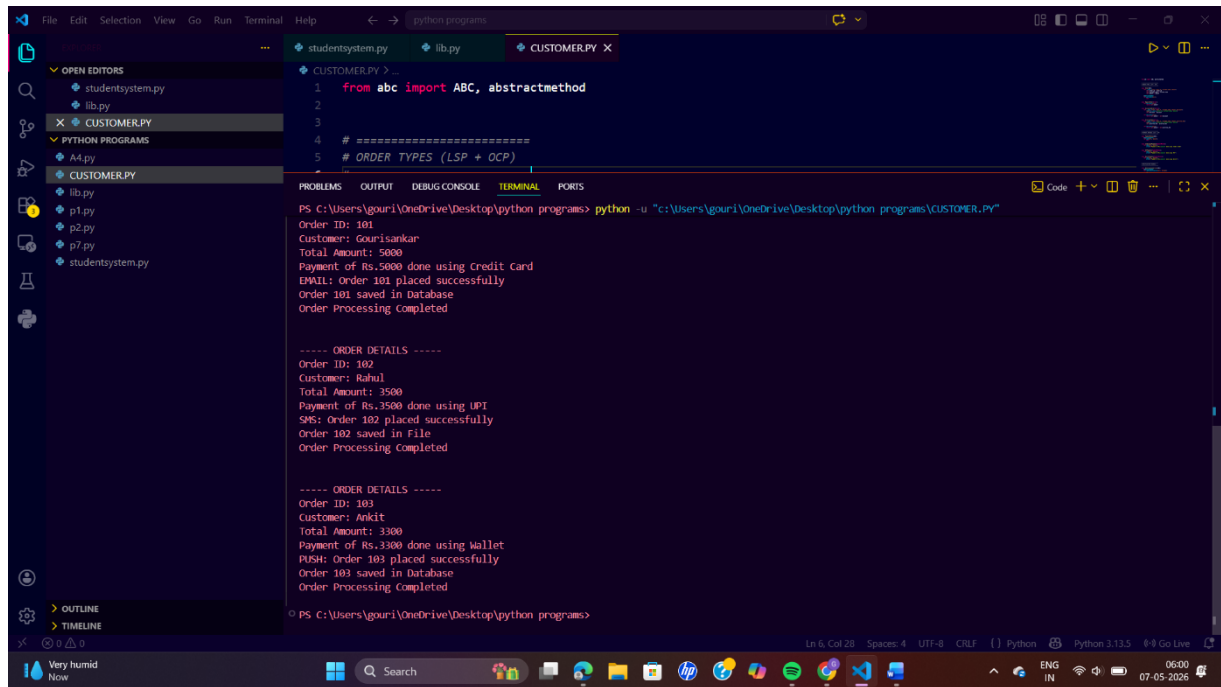
```
service1 = OrderService(
    CreditCardPayment(),
    EmailNotification(),
    DatabaseStorage()
)
```

```
service2 = OrderService(
    UPIPayment(),
    SMSNotification(),
    FileStorage()
)
```

```
service3 = OrderService(
    WalletPayment(),
    PushNotification(),
    DatabaseStorage()
)
```

```
service1.place_order(order1)
service2.place_order(order2)
service3.place_order(order3)
```

SCREENSHOT OF THE OUTPUT:



The screenshot shows a Visual Studio Code editor window with a Python file named `CUSTOMER.PY` open. The file contains a script that generates order details for three different customers. The terminal output shows the execution of the script, displaying the order details for each customer.

```
1 from abc import ABC, abstractmethod
2
3
4 # =====
5 # ORDER TYPES (LSP + OCP)
```

Terminal Output:

```
PS C:\Users\gauri\OneDrive\Desktop\python programs> python -u "c:\Users\gauri\OneDrive\Desktop\python programs\CUSTOMER.PY"
Order ID: 101
Customer: Gourisankar
Total Amount: 5000
Payment of Rs.5000 done using Credit Card
EMAIL: Order 101 placed successfully
Order 101 saved in Database
Order Processing completed

----- ORDER DETAILS -----
Order ID: 102
Customer: Rahul
Total Amount: 3500
Payment of Rs.3500 done using UPI
SMS: Order 102 placed successfully
Order 102 saved in File
Order Processing Completed

----- ORDER DETAILS -----
Order ID: 103
Customer: Ankit
Total Amount: 3300
Payment of Rs.3300 done using Wallet
PUSH: Order 103 placed successfully
Order 103 saved in Database
Order Processing Completed
```

The status bar at the bottom indicates the file is at line 6, column 28, with 4 spaces, UTF-8 encoding, and CRLF line endings. The Python version is 3.13.5.